

Day 2



高校数学と Julia 言語

整数問題をプログラムで考えよう！

テーマ：数学は実験できる

城北中学校・高等学校 中学 3 年・高校 1 年

夏期講習会 III 2026/8/24 ~ 8/28 担当：清水団

5 日間の学習予定 — テーマ「実験する高校数学」



1 Day 1 計算・基本文法

Julia と友だちになる

2 Day 2 整数問題 ← 今日はこちら！

探す

3 Day 3 場合の数・組合せ

全部作る

4 Day 4 関数・グラフ・最大最小

描く

5 Day 5 確率・シミュレーション

試す

今日のゴール

for 文と if 文で整数を
「全部探す」

未解決問題を自分の手
で実験する

約数を全部求めよう



n の約数 = 「 $n \% d == 0$ となる d」を探すだけ！

```
divisors(n) = [d for d in 1:n if n % d == 0]
```

```
divisors(24)
```

```
[1, 2, 3, 4, 6, 8, 12, 24]
```

```
length(divisors(24))    # 約数の個数
```

```
8
```

```
sum(divisors(24))       # 約数の和
```

```
60
```

1 行

「約数を全部求める関数」が
1 行で完成！

[]

内包表記：条件に合う数を
集めてリストにする書き方

- この `divisors` 関数が今日ずっと活躍します

素数判定と Primes.jl



素数 = 約数がちょうど 2 個の数。まず自作、そしてプロの道具へ。

```
# 自作：約数が 2 個なら素数
isprime_mine(n) = n ≥ 2 && length(divisors(n)) == 2

# プロの道具
using Pkg; Pkg.add("Primes"); using Primes
isprime(9999999967)
true
primes(100)      # 100 以下の素数を全部
factor(2026)     # 素因数分解
2 × 1013
```

25 個

100 以下の素数の個数
length(primes(100)) で一発

0.1 秒

100 億に近い数の素数判定も
一瞬で終わる

- 手作りで仕組みを理解 → パッケージで高速に実験

エラトステネスの篩（ふるい）



紀元前 3 世紀のアルゴリズムを、自分の手で実装してみよう

```
function eratosthenes(n)
    is_prime = trues(n); is_prime[1] = false
    for p in 2:n
        if is_prime[p]
            for q in 2p:p:n    # p の倍数を消す
                is_prime[q] = false
            end
        end
    end
    return [p for p in 1:n if is_prime[p]]
end
```

ふるいの手順

- 2 から n まで数を並べる
- 残った最小の数は素数
- その倍数を全部消す
- 繰り返す！

2300 年前の方法が、今もコンピュータの中で現役

- **primes(1000)** と結果が一致するか確認してみよう

素数はどのくらいあるのか？ 実験！



n 以下の素数の個数を数えて、割合の変化を観察しよう

25%

100 以下
素数 25 個

16.8%

1,000 以下
素数 168 個

12.3%

10,000 以下
素数 1,229 個

9.6%

100,000 以下
素数 9,592 個

だんだん「まばら」になるが、なくなる (素数は無限にある!)。この減り方の法則 = 素数定理の発見も、ガウスの計算実験から始まりました。

ユークリッドの互除法：世界最古のアルゴリズム



原理 : a を b で割った余りが r なら $\gcd(a, b) = \gcd(b, r)$

```
gcd(1071, 1029)    # 標準装備
21
lcm(12, 18)        # 最小公倍数
36

function my_gcd(a, b)  # 自分でも書ける！
  while b != 0
    a, b = b, a % b
  end
  return a
end
```

紀元前 300 年

ユークリッド「原論」掲載。
世界最古のアルゴリズムの一つ

$\gcd \times \text{lcm}$

= 2 数の積になることも
実験で確認できる

- **while 文** : 条件が成り立つ間ずっと繰り返す (for の仲間)

完全数を探そう



完全数 = 自分以外の約数の和が自分自身と等しい数 (例: $6 = 1 + 2 + 3$)

```
proper_sum(n) = sum(divisors(n)) - n
```

```
for n in 2:10000
  if proper_sum(n) == n
    println("$n は完全数!")
  end
end
```

```
end
```

```
6    28    496    8128
```

4 個

10000 以下の完全数。
5 個目は 33550336!

未解決

「奇数の完全数はあるか？」は
現在も誰も知らない

- 古代ギリシャ人が愛した数を、自分のプログラムで再発見！

双子素数を探そう



双子素数 = 差が 2 の素数のペア : (3,5), (5,7), (11,13), ...

```
for n in 2:100
    if isprime(n) && isprime(n+2)
        println("($n, $(n+2))")
    end
end
(3, 5) (5, 7) (11, 13) (17, 19)
(29, 31) (41, 43) (59, 61) (71, 73)
```

&&

「かつ」で 2 つの条件を
組み合わせる

未解決 2 つ目

「双子素数は無限にあるか？」
も誰も証明できていない

- 今日だけで未解決問題に 2 つ出会いました

フェルマー数：大数学者の予想が崩れた日



フェルマーの予想 : $F_n = 2^{(2^n)} + 1$ はすべて素数 ($F_0 \sim F_4$ は確かに素数)

```
for n in 0:4
    F = big(2)^(2^n) + 1
    println("F_$n = $F")
end
3  5  17  257  65537  すべて素数

factor(big(2)^(2^5) + 1)  # F5 は？
641 × 6700417  ← 素数ではない！
```

1732 年

オイラーが F_5 の反例を発見。
フェルマーの予想は崩れた

1 秒

オイラーの大発見を、私たちは
1 秒で再現できる

- `big( )` を使うと何百桁の整数も正確に計算できる

コラッツ予想：世界一有名な未解決問題



偶数なら 2 で割る、奇数なら 3 倍して 1 足す → どんな数も必ず 1 になる… ? (1937 年から未解決)

```
function collatz_steps(n)
  steps = 0
  while n != 1
    n = iseven(n) ? n ÷ 2 : 3n + 1
    steps += 1
  end
  return steps
end

argmax(collatz_steps, 1:10000)
6171 ← 一番長い旅をする数
```

実験でわかること

- 27 から始めると 111 ステップの大冒険 (最高 9232 まで上昇!)
- 1 ~ 10000 の最長は 6171 (261 ステップ)
- 2^{68} (約 30 京) まで成立を確認済み

● あなたも今日、未解決問題の「検証」に参加しました!

今日の演習問題



1 完全数

496 が完全数であることを、約数をすべて表示して確かめよう

2 素数

2 桁の素数は何個？ 200 以下の双子素数をすべて表示しよう

3 コラッツ

好きな数（誕生日など）でコラッツの旅を表示。1 ～ 100 で最長の数も見つけよう

4 チャレンジ：カプレカ数

4 桁の数で「大きい順-小さい順」を繰り返すと必ず 6174 に！
`digits(n)` と `sort` で実験しよう

Julia で実験 → 手と頭で解く → チェック → ノートブックを保存して Google Classroom から提出

今日のまとめ



- 約数・素数 for 文と if 文で「全部探す」
- 篩・互除法 古代のアルゴリズムを自分で実装
- 完全数・双子素数・フェルマー数 数の神秘をプログラムで発見
- コラッツ予想 未解決問題も数行で実験できる

数学は「証明」だけでなく「実験」からも始まる。明日は場合の数 —— 樹形図を「全部作って」します！